

ORBIT-X

AI-Native Decision Intelligence Platform

Complete Repository Transformation, Prioritization, and Removal Plan

Repository audited: GitHub main branch, 22 August 2026

Target perception: DATA → Feature Engineering → ML/AI → Context + Metadata → RAG/Agents/Tools → Decision Intelligence → Optimization → Human Review → Monitoring + Feedback

Domain role: Satellite simulation generates realistic operational data, constraints, and failure scenarios. It is the evaluation environment, not the primary product identity.

1. Executive Decision

Do not rebuild ORBIT-X. The repository already contains a strong AI/ML and decisioning base: Cross-Attention, an MLP surrogate, Isolation Forest, SHAP, CP-SAT, hybrid Dense + BM25 RAG, MCP tools, FastAPI, Redis, SQLAlchemy/PostgreSQL support, resilience scenarios, benchmarks, and a React/Three.js interface. The main problem is positioning and architectural emphasis, not lack of technology. The current README presents ORBIT-X first as an orbital resource allocation / astrodynamics / digital-twin platform, which makes the transferable AI/data engineering story harder to see.

The transformation should therefore be a controlled extraction of the reusable AI platform from the domain simulator: make data, ML, context, agents, explainability, optimization, feedback, and observability the primary spine; keep orbital mechanics, TLEs, ISL, conjunctions, and 3D visualization as a contained simulation domain.

Priority	Decision	What to do
P0	Keep and elevate	Cross-Attention, Isolation Forest, SHAP, CP-SAT, RAG, MCP, FastAPI, PostgreSQL/Redis, tests, benchmarks
P0	Reframe	Simulation, physics, TLE, ISL, resilience as the evaluation domain
P0	Refactor	Move AI/data concerns out of the monolithic intelligence namespace into clear data/ml/context/genai/optimization boundaries
P1	Add	Metadata, lineage, dataset discovery, agent grounding, human approval, feedback/evaluation, agent traces
P1	Reduce from hero path	Multi-agent auctions, laser networking, deep physics exposition, 10 Hz globe-first UX
P2	Keep only as support	Advanced physics benchmarks and detailed orbital theory
Delete only if unused	Dead/duplicated code, unsupported demos, duplicate docs, unused UI and stale claims after dependency/reference audit	

2. Repository Audit: What Exists Today

The current GitHub repository has 21 commits and top-level areas for backend, benchmarks, docs, frontend, observability, Kubernetes, workers, Docker, and design/planning documents. The README currently calls the project an “Autonomous Multi-Satellite Mission Allocation, Astrodynamics Physics Engine, Deep Learning Surrogate & 3D WebGL Digital Twin Platform” and lists CP-SAT, Cross-Attention, physics ODEs, Isolation Forest, TreeSHAP, hybrid RAG, MCP, and Three.js as a combined product story. [Repository audit, GitHub main branch.]

2.1 Existing AI/ML strengths to preserve

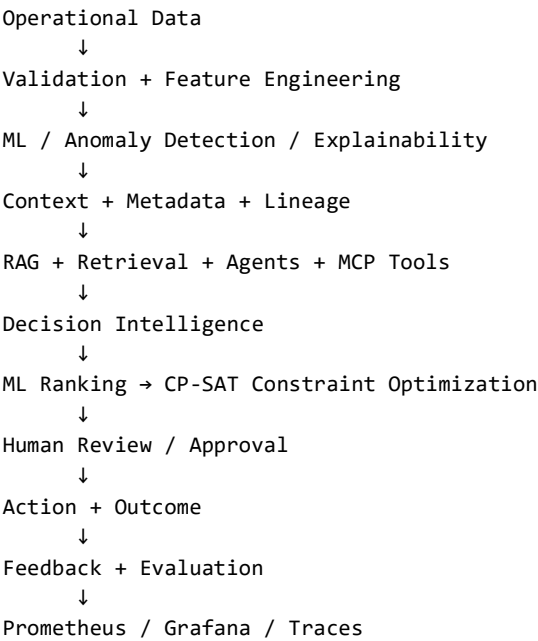
- Cross-Attention network: current implementation models satellite/resource-state tokens against mission-requirement tokens and produces valuation, win-probability, and physics-related outputs.
- BidValueMLP: current neural surrogate / imitation-learning path gives you a simpler baseline against the more advanced attention architecture.
- Isolation Forest health detector: already organized around multivariate telemetry streams and anomaly scoring.
- SHAP explainability: current repository exposes TreeSHAP-style local feature attribution and decision explanation.
- CP-SAT optimizer: current solver is a strong deterministic decision layer after ML scoring/candidate generation.
- Hybrid RAG: current dense + BM25 retrieval is a valuable starting point for context-aware retrieval.
- MCP server: current native tools already expose constellation status, mission explanation, historical RAG, neural preview, and scenario triggering.
- Production foundation: FastAPI, WebSockets, Redis, async SQLAlchemy, PostgreSQL/SQLite fallback, Docker, Kubernetes, tests, evaluation harness, and observability are already present.

2.2 Existing domain/simulation strengths to retain, but demote

- Keplerian/J2 propagation and TLE pipeline.
- Access/elevation visibility model.
- Conjunction / collision logic.
- Inter-satellite laser network model.
- Battery and thermal ODE simulation.
- 10 operational resilience scenarios.

- 3D globe, orbit, laser-mesh, telemetry, schedule, and scenario visualization.

3. Target Architecture



The simulator feeds the bottom of the stack: telemetry, operational states, resource availability, mission requests, constraints, and controlled failure scenarios. This is the bridge that makes the domain useful without making the project dependent on aerospace knowledge.

4. What to REMOVE, REDUCE, MOVE, and KEEP

This section is the most important practical guidance. “Remove” means delete from the main product only after reference checks. “Reduce” means keep the implementation but remove it from the hero workflow and top-level documentation. “Move” means isolate it into the simulation/domain layer. “Keep” means the capability should become more visible.

Area / Current implementation	Action	Reason	Target location
Astrodynamics-heavy README opening	REMOVE from front page; retain content	It immediately positions the project as aerospace/physics rather than AI/data	docs/simulation/ + condensed README subsection
Detailed orbital mechanics equations and assumptions	MOVE	Useful validation material, but not transferable core	docs/simulation/physics.md
Keplerian/J2 propagator	KEEP but demote	Generates realistic state/context for AI evaluation	simulation/orbital/
TLE/CelesTrak pipeline	KEEP but demote	Good data engineering/reliability example when framed as external data ingestion	simulation/orbital/tle/
ISL laser mesh as major product feature	REDUCE	Too domain-specific for AI roles; keep as one constraint/resource dimension	simulation/communication/
Pairwise collision/CAM logic	REDUCE	Useful hard-constraint provider; not main value proposition	simulation/constraints/
Thermal/battery ODEs	MOVE + KEEP	Useful simulator-generated features and constraints; physics details should not dominate	simulation/telemetry/physics/
Multi-agent sealed-bid/Vickrey auction	REDUCE strongly; candidate for removal	It can distract from agentic AI: it is a scheduling heuristic, not the same as an LLM/tool-using agent	optimization/baselines/ or experiments/
3D laser-network visualization	DE-EMPHASIZE	Visually impressive but weak evidence for AI/data engineering by itself	frontend/simulation/
Globe-first dashboard	REDESIGN	AI assistant and decision evidence should be the first screen	frontend/features/ai-assistant/
Scenario director as central UI	REDUCE	Keep scenario engine; expose it under Evaluation/Simulation	frontend/features/evaluation/
AI Lab / fine-tuning studio	KEEP only verified capabilities	Fine-tuning UI can look artificial if it is not backed by reproducible training/evaluation	frontend/features/ml-lab/
“Autonomous” claims without human approval	REMOVE/REWRITE	Target architecture is human-in-the-loop and auditable	README + decision workflow

Area / Current implementation	Action	Reason	Target location
Performance claims not generated by current scripts	REMOVE	Avoid unsupported resume-style numbers	README benchmarks only
Duplicate design/planning documents	CONSOLIDATE	Repository currently contains multiple long design/spec documents; one current target architecture should be authoritative	docs/architecture/
Static scientific plots unrelated to hero workflow	MOVE/ARCHIVE	Keep as experiment evidence, not homepage content	docs/experiments/assets/
Unused dependencies / stale imports / dead routes	DELETE after reference audit	Reduces maintenance noise and makes architecture credible	respective modules
Redundant queue/event technologies if introduced only for keywords	DO NOT ADD	No value unless justified by workload	Keep current minimal operational stack

Important: Do not delete the physics package simply because it is domain-specific. The better move is to make it a dependency of the simulation/evaluation layer, not a peer to the AI platform in the top-level narrative.

5. File-Level Refactoring Direction

The current repository exposes the following intelligence files: agent_loop.py, battery_model.py, bid_value_network.py, commentary_generator.py, cross_attention_network.py, decision_logger.py, explainability.py, health_ai.py, hybrid_mission_rag.py, mission_qa.py, multi_agent.py, optimizer.py, pinn_battery_thermal.py, and shap_explainer.py. The current physics package contains access_model.py, collision.py, isl_network.py, orbit_propagator.py, and tle_pipeline.py. The simulation package contains benchmark.py, scenarios.py, and simulator.py.

Current file	Decision	Refactor destination / purpose
agent_loop.py	KEEP + split	genai/agents/agent_loop.py; separate planning from tool execution
mission_qa.py	KEEP + rename/reframe	genai/rag/ or genai/qa/; generic “operational_qa” behavior
hybrid_mission_rag.py	KEEP + generalize	genai/rag/hybrid_retriever.py
cross_attention_network.py	KEEP	ml/models/cross_attention/model.py
bid_value_network.py	KEEP	ml/models/baselines/bid_mlp.py
health_ai.py	KEEP + generalize	anomaly_detection/models/isolation_forest.py
explainability.py	KEEP + consolidate	ml/explainability/explain.py
shap_explainer.py	KEEP + consolidate	ml/explainability/shap.py
optimizer.py	KEEP	optimization/cp_sat/solver.py
multi_agent.py	REDUCE / isolate	optimization/baselines/auction.py; not the primary AI agent
battery_model.py	MOVE	simulation/telemetry/physics/battery.py
pinn_battery_thermal.py	MOVE	simulation/telemetry/physics/thermal.py; keep only if used to generate/evaluate features
decision_logger.py	KEEP + expand	backend/services/decision_audit.py + lineage hooks
commentary_generator.py	REVIEW	Remove if it generates decorative commentary; keep only if it is a grounded explanation service
physics/access_model.py	MOVE	simulation/orbital/access.py
physics/collision.py	MOVE	simulation/constraints/collision.py
physics/isl_network.py	MOVE	simulation/communication/isl.py
physics/orbit_propagator.py	MOVE	simulation/orbital/propagator.py
physics/tle_pipeline.py	MOVE	simulation/orbital/tle.py
simulation/simulator.py	KEEP + rename	simulation/environment.py; explicit data/constraint generation role
simulation/scenarios.py	KEEP	simulation/scenarios/catalog.py
simulation/benchmark.py	KEEP + move	benchmarks/scheduling/; distinguish domain benchmark from ML benchmark

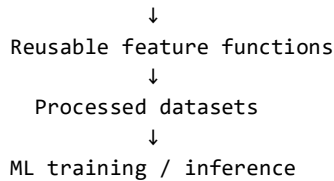
6. Data Engineering Layer

This is the largest missing perception layer. Today the repository has data, TLE cache, simulation, training scripts, and database infrastructure, but the README does not present a clean data lifecycle. Make it explicit.

```

Raw telemetry / mission requests / state / decisions
    ↓
  Schema validation
    ↓
Cleaning + normalization

```



- ☐ Create typed schemas for telemetry, mission/request, resource state, anomaly, prediction, decision, and feedback.
- ☐ Create reusable feature functions instead of embedding transformations inside model training scripts.
- ☐ Persist durable records in PostgreSQL and use Redis for hot operational state/cache.
- ☐ Version feature schemas and model-input schemas.
- ☐ Add basic data-quality checks: nulls, type changes, duplicates, stale timestamps, invalid ranges, and unexpected categories.
- ☐ Create a reproducible dataset-generation command that can regenerate the training/evaluation data from the simulator seed and configuration.

7. ML Engineering Layer

7.1 Model stack

Model	Role	Keep?	Evidence required
Greedy / Random	Operational baseline	KEEP	Completion, reward, latency
BidValueMLP	Simple neural baseline	KEEP	Held-out MAE/agreement/latency
Random Forest / XGBoost	Classical ML baseline	ADD or formalize	Cross-validation + held-out metrics
Cross-Attention	Primary advanced model	KEEP / HERO	Model card + ablation + latency + error analysis
Isolation Forest	Unsupervised anomaly detection	KEEP	Threshold + false positives + detection latency
Hybrid ML + CP-SAT	Final decisioning pattern	KEEP / HERO	Constraint satisfaction + decision quality

7.2 Cross-Attention refactor

The current network already separates satellite-state and mission-requirement tokens and uses multi-head cross-attention. Keep the architecture, but rename the conceptual problem in the documentation to “resource-state/request matching” or “operational resource allocation.” The domain fields can remain concrete in the schema, but the explanation should emphasize the transferable modeling pattern.

- ☐ Create docs/models/cross-attention-model-card.md.
- ☐ Document train/validation/test split and leakage prevention.
- ☐ Document the exact target and losses.
- ☐ Add baseline comparison.
- ☐ Add feature ablation.
- ☐ Add failure cases and calibration/confidence analysis.
- ☐ Report inference latency only with the exact benchmark command and environment.

8. Experiments, Evaluation, and Evidence

The repository currently contains strong benchmark claims, including a six-scheduler suite, Cross-Attention agreement, anomaly-detection results, and scaling numbers. The next step is to turn those into a transparent evidence hierarchy.

- ☐ Create experiments/baseline_comparison/
- ☐ Create experiments/feature_ablation/
- ☐ Create experiments/error_analysis/
- ☐ Create experiments/anomaly_detection/
- ☐ Create experiments/scalability/.
- ☐ Generate metrics from scripts instead of hard-coding README values.
- ☐ Store the command, commit hash, dataset/config version, seed, and environment for every reported experiment.
- ☐ Remove any README number that cannot be reproduced from the current repository.

Evidence	What recruiter should see	Priority
Benchmark table	Model/heuristic vs quality, latency, throughput	P0
Feature ablation	Which features actually matter	P0
Error analysis	Where the model fails and why	P0
SHAP	Why individual predictions happened	P1
Anomaly threshold study	Trade-off between detection and false alarms	P1
Agent evaluation	Tool selection, grounding, retrieval, failure recovery	P1
Load/failure testing	What happens when dependencies fail	P1

9. Anomaly Detection and Explainability

Keep Isolation Forest and SHAP, but connect them to the decision workflow. An anomaly should not just appear in a chart; it should become context that can influence prediction, optimization, and the final explanation.

Telemetry → Feature Extraction → Isolation Forest → Anomaly Score → Context Store → ML Prediction → SHAP → Decision

- ☐ Document telemetry feature set.
- ☐ Document threshold selection.
- ☐ Separate anomaly detection score from business impact.
- ☐ Record anomaly → decision links in lineage.
- ☐ Expose the top contributing features to the human reviewer.
- ☐ Add “why was this resource rejected?” explanations, not only “why was this resource chosen?”

10. Context, Metadata, and Lineage

This is the main new platform layer required to match the target perception.

- ☐ Dataset metadata: schema, version, owner, freshness, quality score, upstream/downstream.
- ☐ Model metadata: version, training dataset, feature schema, metrics, artifact location.
- ☐ Prediction metadata: model version, input record IDs, timestamp, confidence.
- ☐ Decision metadata: candidates, constraints, optimizer version, final action, approval state.
- ☐ Lineage graph: raw data → processed data → features → model → prediction → optimization → decision → outcome.
- ☐ Natural-language data discovery: query metadata first; return actual datasets/fields and evidence.
- ☐ Decision lineage: answer “what data influenced this decision?” and “which models depend on this dataset?”

11. RAG, Agents, MCP, and Grounding

11.1 RAG

Query → Query planner → Metadata search + Dense search + BM25 + SQL → Rerank → Context builder → LLM

- ☐ Keep the existing hybrid Dense + BM25 RAG.
- ☐ Extend retrieval beyond historical mission text into metadata, schema, anomalies, predictions, decisions, and lineage.
- ☐ Return source IDs and timestamps with retrieved facts.
- ☐ Add “insufficient evidence” behavior.
- ☐ Evaluate retrieval separately from generation.

11.2 AI agent

The existing agent/MCP implementation should become the product hero. It must be materially different from the current domain-specific “assistant drawer”: the agent should orchestrate real data and decision capabilities.

User → Intent → Plan → Tools → Evidence → Reasoning → Recommendation → Human approval

- ☐ Standardize tool schemas with Pydantic/JSON-schema-like contracts.

- ☐ Keep tools deterministic and authorization-aware.
- ☐ Record tool inputs/outputs, latency, failures, and trace IDs.
- ☐ Add a read-only data-discovery path before any action-taking tool.
- ☐ Require human approval for high-impact actions.
- ☐ Keep MCP as the external tool protocol; do not make MCP the business logic itself.

12. Hero Workflow: Ask ORBIT-X

Target demo query: Why is Mission M-204 at risk and what should we do?

1. Resolve mission
2. Retrieve operational context
3. Inspect anomalies
4. Get ML prediction
5. Generate SHAP explanation
6. Run/query optimizer
7. Assemble evidence
8. Recommend action
9. Request approval
10. Persist decision + outcome

This should replace a globe-first demo as the primary README GIF/video. The 3D globe can appear in the background or a secondary tab.

13. Human-in-the-Loop and Feedback

- ☐ Add approval state: proposed / approved / rejected / investigate / executed.
- ☐ Persist recommendation, evidence, model version, agent version, reviewer decision, reason, and outcome.
- ☐ Build a feedback dataset from reviewed decisions.
- ☐ Evaluate model, retrieval, and agent changes against reviewed outcomes.
- ☐ Do not auto-retrain from raw feedback; use a validation gate.

14. Backend, Data Stores, Workers, and Infrastructure

14.1 Backend

- ☐ Keep FastAPI as the serving layer.
- ☐ Use Pydantic for external and internal contracts.
- ☐ Organize APIs by data, ML, anomaly, context, agent, decision, feedback, and health domains.
- ☐ Use WebSockets only for genuinely real-time telemetry/agent trace updates; do not make 10 Hz streaming the central product narrative.

14.2 PostgreSQL

- ☐ Durable missions, telemetry summaries, datasets, metadata, models, predictions, anomalies, decisions, feedback, lineage, traces.
- ☐ Add indexes around IDs, timestamps, model versions, and decision relationships.

14.3 Redis

- ☐ Hot telemetry cache.
- ☐ Mission state cache.
- ☐ Agent/session context where needed.
- ☐ Rate limiting / short-lived state.

- ☐ Pub/Sub only for real-time events that truly need it.

14.4 Workers

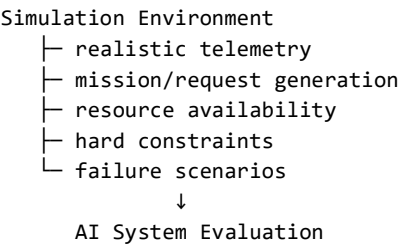
- ☐ Telemetry processing.
- ☐ Anomaly scoring.
- ☐ Embedding/index refresh.
- ☐ Scenario runs.
- ☐ Long optimization jobs.
- ☐ Evaluation jobs.

Do not add Kafka, RabbitMQ, Celery, BullMQ, and another queue together unless one workload specifically justifies it. One simple worker/event approach is stronger than a collection of unused infrastructure badges.

15. Frontend Transformation

Screen	Role	Status after redesign
AI Assistant	Primary user entry point; evidence + tools + recommendation	PRIMARY
Decision Explorer	Candidates, scores, SHAP, constraints, optimizer, final decision	PRIMARY
Data Discovery	Datasets, schema, freshness, quality, lineage	PRIMARY
Agent Trace	Planner, tools, retrieval, evidence, latency	PRIMARY
Monitoring	API/ML/agent/retrieval/data quality metrics	PRIMARY
Simulation	3D globe, orbits, ISL, scenarios	SECONDARY
AI Lab	Training/evaluation controls	SECONDARY; keep only verified functions

16. Simulation as Evaluation Environment



- ☐ Keep physics implementation intact where it supplies data/constraints.
- ☐ Add an explicit “data generation” API or command.
- ☐ Expose scenario seeds and configuration for reproducibility.
- ☐ Generate labeled fault scenarios where possible for evaluation.
- ☐ Treat physics correctness as a simulator validation metric, not the primary AI product metric.

17. Testing, Failure Handling, and Observability

- ☐ Unit tests for feature functions, schemas, tools, optimizer wrappers.
- ☐ Integration tests for PostgreSQL, Redis, FastAPI, RAG, MCP.
- ☐ ML tests for preprocessing consistency, artifact loading, inference contract, metrics.
- ☐ Agent tests for tool choice, invalid tools, missing context, retrieval failure, tool failure, grounding failure.
- ☐ Failure matrix: missing telemetry, stale data, schema drift, model failure, database failure, Redis failure, optimizer timeout, LLM unavailable.
- ☐ Metrics: API latency, model latency, retrieval latency, tool failure rate, anomaly rate, decision approval rate, cache hit rate.
- ☐ Structured logs with request/trace IDs.

18. Documentation Cleanup

The repository currently contains multiple long design/specification documents. Consolidate them so one target architecture is authoritative and the rest are subordinate references.

Document area	Action
README.md	Rewrite completely around AI-native identity
ORBITX_MASTER_ENGINEERING_SPEC.md	Convert into implementation backlog or archive; do not keep conflicting architecture definitions
ORBIT-X_Architecture_and_Design.md	Rewrite as target architecture, or merge into docs/architecture/target-architecture.md
ORBIT-X_Production_Architecture_Complete_Plan.md	Merge useful production items into target architecture and operations docs
walkthrough.md	Rewrite as the new hero workflow walkthrough: Ask ORBIT-X
BASELINE_AUDIT.md	Keep as experiment/evidence document, not homepage content
PHYSICS_ASSUMPTIONS.md	Move under docs/simulation/physics.md
docs/assets/*	Keep benchmark/ML figures; demote thermal/physics figures to simulation docs

19. New README Layout

- ORBIT-X — AI-Native Decision Intelligence Platform
- Hero demo / GIF
- What is ORBIT-X?
- AI-native architecture
- Core capabilities
- ML pipeline + benchmark evidence
- Anomaly detection + SHAP
- Constraint-aware optimization
- Context / metadata / lineage
- RAG + agent + MCP
- Ask ORBIT-X workflow
- Human-in-the-loop + feedback
- Observability
- Production architecture
- Simulation as evaluation domain
- Tech stack
- Quick start
- Testing
- Failure scenarios
- Engineering decisions
- Limitations
- Future work

Opening copy: ORBIT-X is an AI-native decision intelligence platform that combines operational data pipelines, machine learning, anomaly detection, context-aware retrieval, tool-using agents, explainable AI, and constraint-aware optimization to transform operational data into auditable decisions. A satellite simulation environment provides realistic telemetry, constraints, and failure scenarios for evaluating the system.

20. Implementation Roadmap

Phase	Work	Priority	Exit criterion
P0	README + architecture + repository cleanup	Immediate	A recruiter sees AI/data platform identity in first 30–60 seconds
P1	Data schemas + feature pipeline + reproducibility	Immediate	One command regenerates a clean dataset
P2	ML baselines + Cross-Attention + ablation + error analysis	High	Measured comparison and model card exist
P3	Anomaly + SHAP connected to decisions	High	A prediction has both anomaly context and feature explanations

Phase	Work	Priority	Exit criterion
P4	Metadata + lineage + discovery	High	Natural-language data query returns real catalog evidence
P5	RAG + agent + MCP + grounding	High	Agent answers from tools/evidence, with traces
P6	Human approval + feedback	High	Recommendation is reviewable and outcome is persisted
P7	Observability + workers + resilience	Medium	Failures are observable and recoverable
P8	Frontend + demo + README evidence	Medium	One polished end-to-end hero workflow

21. Exact Removal/Reduction Checklist

Use this as the practical cleanup checklist. Do not physically delete a file until a repository-wide reference search shows it is no longer imported, routed, documented, or required by tests.

- ☐ ☐ Remove “Astrodynamics Physics Engine” from the project title and opening sentence.
- ☐ ☐ Remove “Autonomous” from marketing copy unless a human approval boundary and measured autonomy definition are documented.
- ☐ ☐ Remove unsupported sub-millisecond / zero-violation / throughput claims from README if the current benchmark cannot reproduce them.
- ☐ ☐ Remove or collapse repeated STAR/layman explanations from the README once the concise product narrative is established.
- ☐ ☐ Remove detailed space-domain explanations from the first half of README.
- ☐ ☐ Move orbital equations and physical-constant exposition to simulation documentation.
- ☐ ☐ Move TLE cache details to simulation/orbital documentation.
- ☐ ☐ Move ISL laser-network details to simulation/communication documentation.
- ☐ ☐ Move collision/CAM logic to the constraint-provider section.
- ☐ ☐ Remove the auction/Vickrey system from the hero AI narrative; keep only as a scheduler baseline if it is experimentally useful.
- ☐ ☐ Remove decorative commentary generation if it is not grounded in structured evidence.
- ☐ ☐ Remove duplicate architecture documents after merging their unique content.
- ☐ ☐ Remove stale UI tabs/components that do not map to a supported backend capability.
- ☐ ☐ Remove unused dependencies and dead routes after import/reference analysis.
- ☐ ☐ Remove any “research platform” wording that makes the transferable engineering less obvious.
- ☐ ☐ Do not delete physics code that produces telemetry, constraints, or evaluation scenarios.

22. Final Capability Matrix

Capability	Final role	Priority	Primary evidence
Data pipeline	Operational data foundation	P0	schemas + reproducible dataset command
Feature engineering	Reusable ML inputs	P0	feature modules + tests
Cross-Attention	Primary learned ranking model	P0	model card + benchmarks + ablation
XGBoost / classical ML	Baseline comparison	P0/P1	held-out evaluation
Isolation Forest	Anomaly detection	P1	threshold + fault analysis
SHAP	Decision explainability	P1	local/global explanations
CP-SAT	Constraint decision layer	P1	constraint satisfaction + outcome metrics
Metadata	Context system	P1	dataset/model metadata API
Lineage	Auditability	P1	data → model → decision graph
RAG	Evidence retrieval	P1	retrieval evaluation + citations
Agents	Workflow orchestration	P1	tool traces + grounded responses
MCP	Tool protocol	P1	typed tools and server integration
Human approval	Safe decision boundary	P1	approval/rejection history
Feedback	Continuous evaluation	P1	reviewed outcomes dataset
PostgreSQL	Durable state	P0	migrations + indexes
Redis	Operational cache/state	P0	cache metrics + TTL policy
FastAPI	Serving interface	P0	API + integration tests
Prometheus/Grafana	Monitoring	P1	dashboards + metrics
Kubernetes	Deployment depth	P2	working manifests
Three.js	Domain visualization	P2	secondary simulation screen
Advanced physics	Evaluation domain	P3	simulation validation only

23. Final Interview Positioning

AI Engineer

I built an AI decision intelligence platform that combines operational ML, anomaly detection, SHAP explainability, hybrid RAG, tool-using agents, MCP, FastAPI serving, and constraint-aware optimization. The agent operates over real system capabilities and evidence rather than acting as a standalone chatbot.

ML Engineer

I built a reproducible ML pipeline with classical baselines, a Cross-Attention model, anomaly detection, feature ablation, error analysis, explainability, model serving, and evaluation against a deterministic optimizer.

Data / AI Platform

I added structured metadata, lineage, data discovery, RAG, tool orchestration, PostgreSQL/Redis state, observability, human review, and feedback so model outputs can be traced to data and decisions.

When asked about the satellite domain

The satellite simulator is the evaluation environment. It generates realistic telemetry, operational states, constraints, and controlled failures. The transferable engineering is the data/ML/context/agent/decision platform built around that environment.

24. Definition of Done

- ☐ ☐ README is AI-first and domain-neutral in the first section.
- ☐ ☐ Physics is secondary and isolated in simulation docs/modules.
- ☐ ☐ Data ingestion, validation, features, and dataset generation are reproducible.
- ☐ ☐ Cross-Attention is formally evaluated against baselines.
- ☐ ☐ Feature ablation and error analysis exist.
- ☐ ☐ Isolation Forest and SHAP are connected to decisions.
- ☐ ☐ ML + CP-SAT hybrid flow is explicit.
- ☐ ☐ Metadata, discovery, and lineage are implemented.
- ☐ ☐ RAG is grounded in actual platform data.
- ☐ ☐ Agent calls typed tools and exposes evidence.
- ☐ ☐ MCP exposes the supported tool contract.
- ☐ ☐ Ask ORBIT-X is the hero workflow.
- ☐ ☐ Human approval and feedback are persisted.
- ☐ ☐ Agent traces and system metrics are observable.
- ☐ ☐ PostgreSQL and Redis have intentional responsibilities.
- ☐ ☐ Failure scenarios are tested.
- ☐ ☐ Docker/CI and core deployment path work.
- ☐ ☐ Frontend leads with AI Assistant / Decision Explorer rather than the 3D globe.
- ☐ ☐ Unsupported performance claims are removed.
- ☐ ☐ The project can be explained without aerospace prerequisites.

25. Repository Audit Basis

Primary source: <https://github.com/Susil-commits/ORBIT-X---Autonomous-Orbital-Resource-Intelligence-Network>

The audit reviewed the repository landing page, README, current backend/intelligence tree, backend/physics tree, backend/simulation tree, frontend/src tree, and the repository structure/claims visible on the main branch as of 22 August 2026.

Examples of directly observed repository capabilities include the intelligence modules and MCP server, the physics and simulation packages, the six-scheduler/scale evaluation paths, and the frontend components such as AI Lab, Explainability, Mission RAG, MultiAgent, Scenario Director, GlobeView3D, ScheduleGantt, and Telemetry HUD.